

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE NAME: COMPUTER ARCHITECTURE

COURSE CODE: CSA1206

COURSE OUTCOME COVERED

CO1: Analyze the functional units of a computer system including CPU, memory, bus structures, and I/O components by examining instruction execution cycles, addressing modes, and performance metrics such as CPI, clock cycles, and benchmarking (BL4) Assessment Tool
Weightage: 40%

QUESTIONS:5

Total Marks:25

ANNEXURE A APPLICATION BASED QUESTIONS

Code of Conduct:

I Bharathi.Kadiveti (192412132)(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: bharathi kadiveti

1. A smart city surveillance system processes continuous video feeds from multiple cameras, where large volumes of data are transferred between I/O devices, memory, and CPU. During peak hours, the system experiences lag and delayed response in threat detection. Analyze how the interaction between CPU, memory, and I/O contributes to this issue, and explain how different bus structures (single-bus vs multi-bus) and improvements in instruction execution cycle can enhance system performance.
2. A real-time stock trading application running on a processor executes millions of instructions per second, but users report delays in transaction processing during high load. The system has a high CPI due to frequent memory access and branch instructions. Examine how the fetch–decodeexecute cycle impacts execution time in this scenario, and propose methods to reduce CPI and improve overall CPU performance using suitable architectural enhancements.
3. An embedded system based on an 8085 microprocessor is used in an industrial temperature control unit, where sensors provide input and actuators respond accordingly. However, incorrect temperature readings are occasionally processed due to improper use of addressing modes in the program. Discuss how different addressing modes influence data access, identify possible causes of such errors, and suggest how the instruction set of 8085 can be effectively used to ensure accurate and reliable operation.
4. A computing system uses an 8086 microprocessor with segmented memory for executing multiple programs simultaneously, but it encounters issues like incorrect data access and stack overflow errors. Evaluate how segmentation (code, data, and stack segments) works in 8086, analyze how improper segment handling can lead to such problems, and explain how instruction execution and addressing modes must be managed to ensure correct program execution.

5. A data communication system represents packet information in hexadecimal format for ease of debugging and transmission, but a software engineer mistakenly interprets values, leading to incorrect data processing. Analyze the importance of number system conversions between hexadecimal and binary in such systems, explain how errors in conversion can affect instruction execution, and discuss how efficient CPU design and benchmarking can help detect and minimize such issues in real-time systems.

CO1-ASSESSMENT TOOL 1

Bharathi.kadiveti (192412132)

Q1. Smart City Surveillance System – CPU, Memory and I/O Performance

Introduction

A smart city surveillance system continuously collects video from multiple cameras. The video data is transferred between input/output (I/O) devices, memory, and the CPU. During peak hours, a large amount of data is processed simultaneously, causing delays in threat detection. This occurs because the CPU, memory, and I/O devices compete for system resources.

Interaction between CPU, Memory and I/O

- CPU: Executes instructions and analyzes video frames.
- Memory (RAM): Temporarily stores video data before processing.
- I/O Devices: Cameras continuously send high-resolution video to the system.

When many cameras transmit data simultaneously:

- Memory becomes overloaded.
- CPU waits for data from memory.
- I/O devices generate more data than the CPU can process.

- This creates a bottleneck, increasing response time.

Effect of Bus Structure

Single Bus Architecture

- CPU, memory, and I/O share one communication bus.
- Only one data transfer occurs at a time.
- Heavy traffic causes bus contention.
- System performance decreases.

Advantages

- Simple design
- Low cost

Disadvantages

- Slow communication
- High waiting time
- Poor scalability

Multi-Bus Architecture

- Uses separate buses for CPU-memory and I/O communication.
- Multiple data transfers occur simultaneously.
- Reduces bus congestion.
- Improves throughput.

Advantages

- Faster communication

- Better multitasking
- Suitable for large surveillance systems

Improving Instruction Execution Cycle

The instruction cycle consists of:

1. Fetch
2. Decode
3. Execute
4. Memory Access
5. Write Back

Performance improvements include:

- Instruction pipelining
- Cache memory (L1, L2, L3)
- Direct Memory Access (DMA)
- Branch prediction
- Multi-core processors
- Parallel processing

These techniques reduce CPU waiting time and increase processing speed.

Conclusion

Using a multi-bus architecture, cache memory, DMA, and instruction pipelining significantly improves surveillance system performance. These enhancements reduce lag, minimize CPU idle time, and enable faster threat detection in real-time applications.

Q2. Real-Time Stock Trading System – High CPI Analysis

Introduction

A real-time stock trading application executes millions of instructions every second. During heavy trading, users experience transaction delays because the processor has a high Cycles Per Instruction (CPI) due to frequent memory accesses and branch instructions.

Fetch–Decode–Execute Cycle

1. Fetch

- CPU fetches instructions from memory.

2. Decode

- The Control Unit decodes the instruction.

3. Execute

- ALU performs calculations or comparisons.

4. Memory Access

- Data is read or written to memory.

5. Write Back

- Results are stored in registers.

When memory access is slow or branches occur frequently, the CPU waits, increasing CPI.

Reasons for High CPI

- Cache misses
- Frequent memory access
- Branch instructions
- Pipeline stalls
- Data hazards
- Instruction dependencies

Methods to Reduce CPI

Cache Memory

- Stores frequently used instructions.
- Reduces memory access time.

Instruction Pipelining

- Executes multiple instruction stages simultaneously.
- Improves CPU utilization.

Branch Prediction

- Predicts branch outcomes.
- Reduces pipeline flushing.

Out-of-Order Execution

- Executes independent instructions first.
- Minimizes waiting.

Multi-Core Processors

- Distributes workload among multiple cores.
- Improves throughput.

Prefetching

- Loads future instructions before they are needed.

Result

These architectural enhancements:

- Lower CPI
- Increase instruction throughput
- Reduce transaction delays
- Improve real-time stock trading performance

Conclusion

Efficient CPU architecture using cache memory, instruction pipelining, branch prediction, prefetching, and multi-core processing minimizes CPI and ensures faster and more reliable transaction processing. Instruction pipelining allows multiple instructions to be processed simultaneously at different stages of execution, increasing the number of instructions completed per clock cycle. Branch prediction helps the processor predict the outcome of branch instructions, reducing pipeline stalls and improving execution efficiency. Prefetching loads instructions and data into the cache before they are required, which further decreases CPU waiting time. Additionally, multi-core processors distribute the workload among multiple processing cores, enabling parallel execution of trading transactions and improving overall system throughput.

Q4. 8086 Segmentation and Stack Overflow

Introduction

The 8086 microprocessor is a 16-bit processor capable of addressing up to 1 MB (2^{20} bytes) of memory. Since its internal registers are only 16 bits wide, it cannot directly generate a 20-bit memory address. To overcome this limitation, the 8086 uses a segmented memory architecture, where memory is divided into logical segments. Each segment can store different types of information such as program instructions, data, and stack. Segmentation improves memory organization, allows efficient program execution, supports modular programming, and enables multiple programs to run more effectively.

Memory Segments in 8086

The 8086 divides memory into four main segments, each identified by a segment register.

1. Code Segment (CS)

The Code Segment (CS) contains the executable instructions of the program. The CPU fetches instructions from this segment during execution. It works together with the Instruction Pointer (IP), which stores the offset address of the next instruction to be executed.

Functions

- Stores machine instructions.
- Controls the sequence of program execution.
- Works with IP to locate instructions.

2. Data Segment (DS)

The Data Segment (DS) stores all program data such as variables, constants, arrays, and tables. Most arithmetic and logical operations access data from this segment.

Functions

- Stores variables and constants.
- Used for arithmetic and logical operations.
- Provides data for program execution.

3. Stack Segment (SS)

The Stack Segment (SS) is used for temporary storage during program execution. It stores return addresses, local variables, parameters passed to functions, and CPU register values during interrupts or subroutine calls. It works together with the Stack Pointer (SP).

Functions

- Stores return addresses.
- Saves register contents.
- Stores local variables and function parameters.
- Supports function calls and interrupts.

4. Extra Segment (ES)

The Extra Segment (ES) is an additional data segment mainly used during string operations and for storing extra data that does not fit into the Data Segment.

Functions

- Provides additional storage space.
- Used in string manipulation instructions.

- Supports efficient data transfer.

Working of Segmentation

The 8086 generates a 20-bit physical address by combining the segment address with the offset address.

Formula

$$\text{Physical Address} = \text{Segment} \times 10\text{H} + \text{Offset}$$

The segment address is shifted left by 4 bits (multiplied by 16 or 10H), and then the offset address is added.

Example

Segment Address = **2000H**

Offset Address = **1500H**

Physical Address

$$= 2000\text{H} \times 10\text{H} + 1500\text{H}$$

$$= 20000\text{H} + 1500\text{H}$$

$$= \mathbf{21500\text{H}}$$

Using this method, the 8086 can access the complete **1 MB memory space** even though it uses only 16-bit registers.

Problems Due to Improper Segment Handling

If segment registers are not initialized correctly or the wrong segment is accessed, several errors can occur.

1. Incorrect Data Access

Incorrect data access occurs when the Data Segment (DS) points to an incorrect memory location. As a result, the processor reads or writes data from the wrong address.

Effects

- Incorrect variable values.
- Wrong arithmetic or logical calculations.
- Corrupted data.
- Unpredictable program behavior.
- Program execution errors.

2. Stack Overflow

A stack overflow occurs when too many PUSH instructions are executed without corresponding POP instructions. The stack grows beyond its allocated memory space and overwrites other memory locations.

Effects

- Program crash.
- Corruption of stack contents.
- Loss of return addresses.
- Incorrect function execution.
- Unexpected program termination.

Managing Addressing Modes

The 8086 supports different addressing modes to access operands efficiently. Selecting the correct addressing mode improves execution speed and prevents memory errors.

Immediate Addressing

The operand value is directly included in the instruction.

Example

```
MOV AX, 1234H
```

Register Addressing

The operand is stored in a CPU register.

Example

```
MOV AX, BX
```

Direct Addressing

The instruction directly specifies the memory address.

Example

```
MOV AX, [2500H]
```

Register Indirect Addressing

The memory address is stored inside a register such as BX or SI.

Example

```
MOV AX, [BX]
```

Based Addressing

Uses a base register (BX or BP) with an optional displacement.

Example

```
MOV AX, [BX+10]
```

Indexed Addressing

Uses an index register (SI or DI) to access array elements.

Example

```
MOV AX, [SI]
```

Based Indexed Addressing

Combines both base and index registers for flexible memory access.

Example

```
MOV AX, [BX+SI]
```

Using the appropriate addressing mode reduces execution time, improves memory utilization, and ensures accurate data access.

Preventive Measures

To avoid segmentation errors and stack overflow, the following practices should be followed:

- Initialize all segment registers (CS, DS, SS, and ES) correctly before program execution.
- Allocate sufficient memory for the stack according to application requirements.
- Ensure every **PUSH** instruction has a corresponding **POP** instruction to maintain stack balance.
- Validate memory addresses before reading or writing data.
- Select appropriate addressing modes for efficient memory access.
- Monitor stack usage during debugging to detect overflow conditions.

Conclusion

The **8086 segmented memory architecture** allows the processor to efficiently access **1 MB of memory** using 16-bit registers by dividing memory into **Code, Data, Stack, and Extra Segments**. Proper management of these segments is essential for reliable and error-free program execution. Incorrect segment initialization, improper addressing modes, or poor stack management can lead to data corruption, stack overflow, invalid memory access, and system crashes. By correctly initializing segment registers, balancing **PUSH** and **POP** operations, selecting suitable addressing modes, and regularly debugging memory usage, the performance and reliability of the 8086 microprocessor can be significantly improved. Proper segmentation ensures efficient execution of programs, better memory organization, and stable operation in multitasking and embedded system applications.

